Arc Pump — API Specification

Pay-per-call HTTP service for autonomous AI agents, settled in native USDC on Arc.

`OpenAPI 3.1 · v1.0.0 · Arc Testnet (eip155:5042002)`
`https://agentpay-service.arcpump2403.workers.dev · Nguyen Phuoc Hau · sieusayza@gmail.com`

OVERVIEW

Payment follows the x402 pattern. An unpaid request returns `402 Payment Required` with an invoice; the agent settles that invoice on-chain; the same request retried with the invoice id returns `200` with the resource. No API keys, no accounts, no human in the loop.

> **Why a settlement contract.** On Arc, USDC is the *native* token, so a plain value transfer carries no memo and cannot be tied to an invoice. Payment therefore goes through a small non-custodial `PaymentRouter`, which binds each payment to both the invoice id *and* the recipient, so the service can verify it in a single `eth_call`. The router forwards funds in the same transaction and never custodies them.

PAYMENT TERMS

Field	Value
Protocol	`x402 v2`
Scheme	`arc-router-v1`
Network	`eip155:5042002 (Arc Testnet)`
Asset	Native USDC, 18 decimals
Price per call	`0.01 USDC`
Pay to	`0xfC6153A6d0Cc40E17d9B48fE2fb1AACd9C63114e`
Settlement contract	`0x42bCE0940b286b29A7bE50c3C7c89302A48E28ff`
Invoice lifetime	900 seconds, single-use
Custody	None — value forwarded to payee in the same transaction

On the scheme name. The advertised scheme is `arc-router-v1` rather than `exact`. `exact` settles an EIP-3009 signed authorization, and native USDC has nothing to authorize against. Declaring a distinct scheme lets a standard x402 client read the terms and skip cleanly, instead of attempting a payment that could not succeed.

ENDPOINTS

GET /fairvalue — `paid, 0.01 USDC`

Prices every live Polymarket "Up or Down" market ending in the next 30 minutes against live spot, and returns the gap between the model and the order book. Not the odds — anyone can read those off the book — the *mispricing*.

Zero-drift lognormal: `fair = Φ( ln(spot/open) / (σ·√minutes_left) )`, with σ estimated from the last 60–90 one-minute log returns. A market whose period has not opened yet has no reference price, so its fair value is exactly 0.5 and it is returned as `status: pending` rather than priced off a stale candle.

Each row returns `market, asset, status, periodOpen, spot, sigmaPerMin, minutesLeft, fairValue, bestBid, bestAsk, edge, signal`, sorted by edge. `signal`

is `BUY_YES` / `BUY_NO` when the gap is at least 5 cents with 50s or more remaining, otherwise `WAIT`.

> **On data sources — two corrections worth stating.** Prices come from the Polymarket **CLOB order book**. Gamma's `bestBid` / `bestAsk` fields do not match the book: a market quoting 0.130/0.131 on the CLOB returned 0.03/0.04 from Gamma, which would manufacture double-digit "edges" that do not exist. Spot comes from **Bybit**, because Binance answers 403 to Cloudflare Worker egress IPs.

GET /premium — `paid, 0.01 USDC`

Call without `invoice` to receive a 402 and an invoice. Settle it, then call again with `?invoice=<id>`. The invoice id may also be sent as the `x-payment` header. Optional `x-agent` and `x-agent-name` headers are recorded to the public ledger.

Status	Meaning
200	Payment verified on-chain; resource served; invoice now spent
402	No invoice supplied (a new one is issued), or supplied invoice not yet paid
400	Unknown or expired invoice
409	Invoice already used — invoices are single-use

Every 402 also carries the standard x402 `PAYMENT-REQUIRED` response header (base64-encoded JSON), exposed via CORS:

```
HTTP/1.1 402 Payment Required
PAYMENT-REQUIRED: <base64 JSON>
access-control-expose-headers: PAYMENT-REQUIRED

{
  "x402Version": 2,
  "error": "Payment required",
  "resource": {
    "url": "https://agentpay-service.arcpump2403.workers.dev/premium",
    "description": "Pay-per-call resource settled in native USDC on Arc.",
    "mimeType": "application/json"
  },
  "accepts": [{
    "scheme": "arc-router-v1",
    "network": "eip155:5042002",
    "amount": "10000000000000000",
    "asset": "0x0000000000000000000000000000000000000000",
    "payTo": "0xfC6153A6d0Cc40E17d9B48fE2fb1AACd9C63114e",
    "maxTimeoutSeconds": 900,
    "extra": {
      "name": "USDC", "decimals": 18, "native": true,
      "router": "0x42bCE0940b286b29A7bE50c3C7c89302A48E28ff",
      "invoiceId": "0x…",
      "pay": "router.pay(bytes32 invoiceId, address service) payable",
      "verify": "router.verify(bytes32,address,uint256) view returns (bool)",
      "retry": "https://…/premium?invoice=0x…"
    }
  }]
}
```

GET /ledger — `free`

Public ledger of every payment this service has verified on-chain, newest first, with aggregate stats. Keyless. Returns `{ purchases[], stats }`.

SETTLEMENT CONTRACT

```
function pay(bytes32 invoiceId, address payable service) external payable;
    // Records paidAmount[invoiceId] and paidTo[invoiceId], forwards msg.value
```

```
    // to `service`, emits Paid. Reverts on zero amount or double-pay.

function verify(bytes32 invoiceId, address service, uint256 minAmount)
    external view returns (bool);
    // One-call check a service makes before releasing its response:
    // paidTo[invoiceId] == service && paidAmount[invoiceId] >= minAmount.
```

Deployed and verifiable at
`testnet.arcscan.app/address/0x42bCE0940b286b29A7bE50c3C7c89302A48E28ff`

FULL FLOW

```
# 1 — unpaid request returns 402 + invoice
curl -i https://agentpay-service.arcpump2403.workers.dev/premium

# 2 — settle on Arc: router.pay(invoiceId, service) with 0.01 USDC as value

# 3 — retry with the invoice id; verified on-chain, served once
curl "https://agentpay-service.arcpump2403.workers.dev/premium?invoice=0x…"
```

NOTES FOR REVIEWERS

- `/fairvalue` is the product: live data, real model, proven end to end on-chain. `/premium` is kept only as the original demonstration endpoint and returns a placeholder payload.
- The gate runs the data producer *before* burning the invoice, so if an upstream source is down the buyer keeps their paid invoice. This is not theoretical — it happened during build-out when Binance began refusing Worker egress, and the invoice paid before the outage was honoured after the fix.
- The rail was built independently before Circle Gateway Nanopayments was found. The design converges on the same problem and the same chain; Gateway's batched EIP-3009 settlement scales further into sub-cent territory, which on-chain-per-call settlement cannot reach. Migration to Gateway Nanopayments is of interest.
- An interactive demo, where a single click triggers a real on-chain payment, is at `arcpump.com/pay`.

Arc Pump · OpenAPI 3.1 machine-readable spec: github.com/PHUOCHAU2403/arc-pump — agentpay/service/openapi.yaml
Finalist, Encode Club × Remix AI Bootcamp for Real World Impact 2026.